

Instructions:

You will work on these exercises during the practical sessions in the fourth week of the course.

Below is an indication of the exercises that we will cover (at least) in each session. We encourage you try them before the session.

- First session (Wednesday/Thursday): exercises 1 and 3.
- Second session (Friday): exercise 2.

Exercise 1: Multi-class Logistic Regression

In the binary case, logistic regression models posterior class probabilities with a sigmoid. For $K > 2$ classes, the posterior probabilities are modeled with the softmax function:

$$y_k = p(\mathcal{C}_k | \phi) = \frac{\exp(a_k)}{\sum_{i=1}^K \exp(a_i)},$$

where

$$a_k = \mathbf{w}_k^\top \phi, \quad \phi = \phi(\mathbf{x}) = (\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x}))^\top,$$

with $\phi_0(\mathbf{x}) = 1$. Note that y_k depends on all class weight vectors $\mathbf{w}_1, \dots, \mathbf{w}_K$ through the denominator of the softmax.

The training set consists of a design matrix Φ and a target matrix $\mathbf{T}$. Each row of $\mathbf{T}$ is one-hot encoded: the n -th row is $\mathbf{t}_n^\top = (t_{n1}, \dots, t_{nK})$, where $t_{nk} = 1$ if data point n belongs to class $\mathcal{C}_k$ and $t_{nk} = 0$ otherwise.

- Derive $\frac{\partial y_k}{\partial \mathbf{w}_j}$. Use the indicator function δ_{pj} , also called the Kronecker delta δ_{kj} .
- Write down the likelihood $p(\mathbf{T} | \Phi, \mathbf{w}_1, \dots, \mathbf{w}_K)$ as a product over N and K . Use the entries of $\mathbf{T}$ as selectors of the correct class. Then write down the log-likelihood.
- Derive the gradient of the log-likelihood with respect to $\mathbf{w}_j$.
- What objective function do we minimize that is equivalent to maximizing the log-likelihood?
- Write a stochastic gradient algorithm for logistic regression using this objective function. Include indices for time and define the learning rate.
- In practice, vanilla stochastic gradient descent can be inefficient. Point out a potential weakness of the algorithm and suggest a possible improvement.

Exercise 2: Backpropagation

Consider a two-layer neural network defined as follows:

$$\begin{aligned} \mathbf{z}^{(1)} &= \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}, & \mathbf{a}^{(1)} &= \tanh(\mathbf{z}^{(1)}), \\ \mathbf{z}^{(2)} &= \mathbf{W}^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)}, & \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z}^{(2)}), \end{aligned}$$

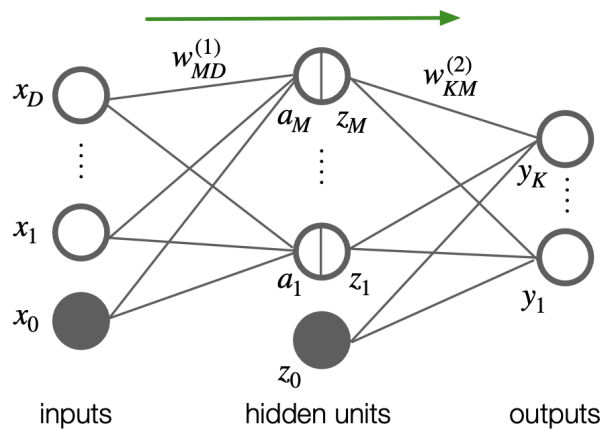
where $\mathbf{x} \in \mathbb{R}^D$ is a single input example, $\hat{\mathbf{y}} \in \mathbb{R}^K$ is the predicted output vector, and the hidden layer has H neurons.

- What are the shapes of $\mathbf{W}^{(1)}$, $\mathbf{b}^{(1)}$, $\mathbf{a}^{(1)}$, $\mathbf{W}^{(2)}$, and $\mathbf{b}^{(2)}$?
- We feed the neural network with a sample $\mathbf{x}$, obtaining the prediction $\hat{\mathbf{y}}$. We assume we have the one-hot ground-truth label $\mathbf{y}$ for that sample. Write down the cross-entropy loss $L(\mathbf{y}, \hat{\mathbf{y}})$ for that sample.
- Compute $\frac{\partial L}{\partial \hat{y}_i}$.
- Compute $\frac{\partial L}{\partial z_i^{(2)}}$.
- Compute $\frac{\partial L}{\partial W_{ij}^{(2)}}$.

Exercise 3: MLP for regression and classification

Exam question

Consider the following Multilayer Perceptron (MLP) architecture:



The figure shows a two-layer MLP architecture. Note that x_0 and z_0 are bias units, which are always equal to 1.

- How many trainable parameters does the model from the figure have?
 - $DM + MK$
 - $D + K^2$
 - $D(D + 1) + K(K + 1)$
 - $M(D + 1) + K(M + 1)$
 - $D(M + 1) + M(K + 1)$
- We want to use the architecture sketched in the figure for a classification task, where we receive an input vector $\mathbf{x} \in \mathbb{R}^D$ and want to classify it into one out of K different classes. We have a dataset of N samples with their corresponding ground-truth labels.
 - What would we need to modify or add to our current architecture to turn it into a classification model that outputs a class label?
 - What loss function should we optimize?

Note: We are expecting a brief list of necessary layers/functions, indicating the name of the function and what its input would be. You do not need to write any formulas.

(c) We want to use the architecture sketched in the figure for a regression task, where we receive an input vector $\mathbf{x} \in \mathbb{R}^D$ and want to predict a single scalar target $t \in \mathbb{R}$. We have a dataset of N samples with their corresponding ground-truth target values.

- (i) What would we need to modify or add to our current architecture to turn it into a regression model that outputs a scalar target?
- (ii) What loss function should we optimize?

Note: We are expecting a brief list of necessary layers/functions, indicating the name of the function and what its input would be. You do not need to write any formulas.

Exercise 4: Cross Entropy

Exam question

We are building an animal classifier. The classifier takes a picture of an animal and outputs a probability distribution over the set of possible animals. Assume that we have three classes: cat, dog, and fish. Given a picture of a cat, the model could output a vector such as

$$\mathbf{q} = (0.7, 0.2, 0.1)^\top.$$

This means that the model assigns the picture a 70% probability of being a cat, 20% probability of being a dog, and 10% probability of being a fish.

To measure how well the model is performing, we use the cross-entropy loss. It compares two probability distributions $\mathbf{p}$ and $\mathbf{q}$, where $\mathbf{p}, \mathbf{q} \in \mathbb{R}^n$, and is defined as

$$L(\mathbf{p}, \mathbf{q}) = - \sum_{i=1}^n p_i \log(q_i).$$

In the example above, $n = 3$, the true distribution is

$$\mathbf{p} = (1, 0, 0)^\top,$$

and the predicted distribution is

$$\mathbf{q} = (0.7, 0.2, 0.1)^\top.$$

We assume that $\mathbf{p}$ is always one-hot encoded, meaning that all entries are zero except for one entry equal to one. In this exercise, use the convention $0 \log(0) = 0$, which is natural because $\lim_{x \rightarrow 0^+} x \log x = 0$.

- (a) What is the value of $L(\mathbf{p}, \mathbf{q})$ for this example?
- (b) Prove that $L(\mathbf{p}, \mathbf{q}) \geq 0$ for all probability distributions $\mathbf{p}, \mathbf{q} \in \mathbb{R}^n$.
- (c) When does $L(\mathbf{p}, \mathbf{q}) = 0$? Remember that $\mathbf{p}$ is one-hot encoded.
- (d) When is $L(\mathbf{p}, \mathbf{q})$ maximized? Remember that $\mathbf{p}$ is one-hot encoded.